



# Manual De Uso Para La API De AGW

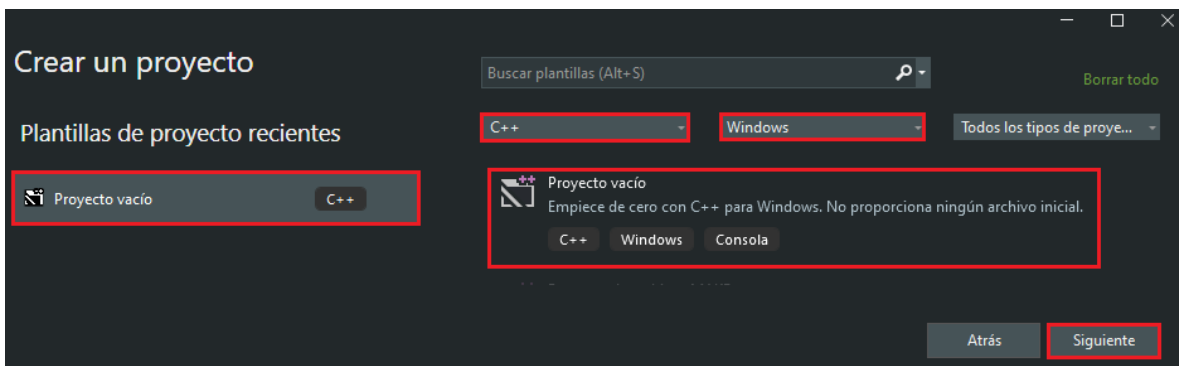
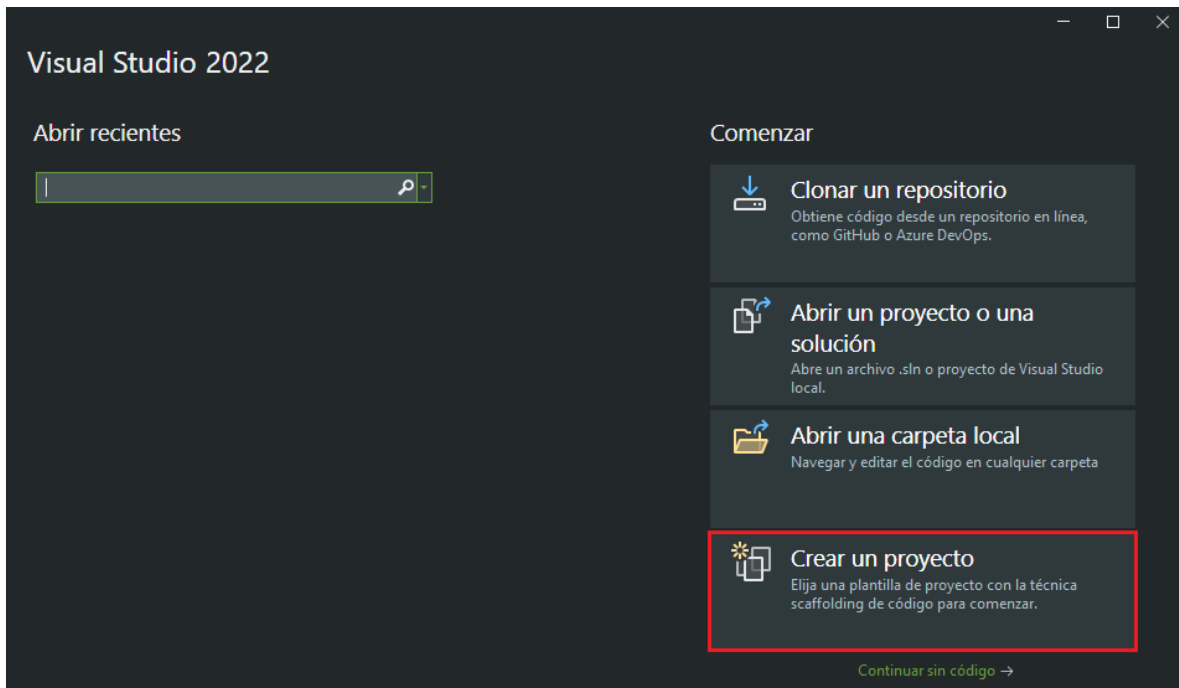
# Índice

|                                         |       |
|-----------------------------------------|-------|
| Inicio básico                           | 2-5   |
| Rutina de inicio                        | 6     |
| Macros de inclusión de contenido        | 7     |
| Manejo de subrutinas de renderizado     | 8     |
| Registro y modificación de deslizadores | 9     |
| Registro y modificación de botones      | 10-11 |

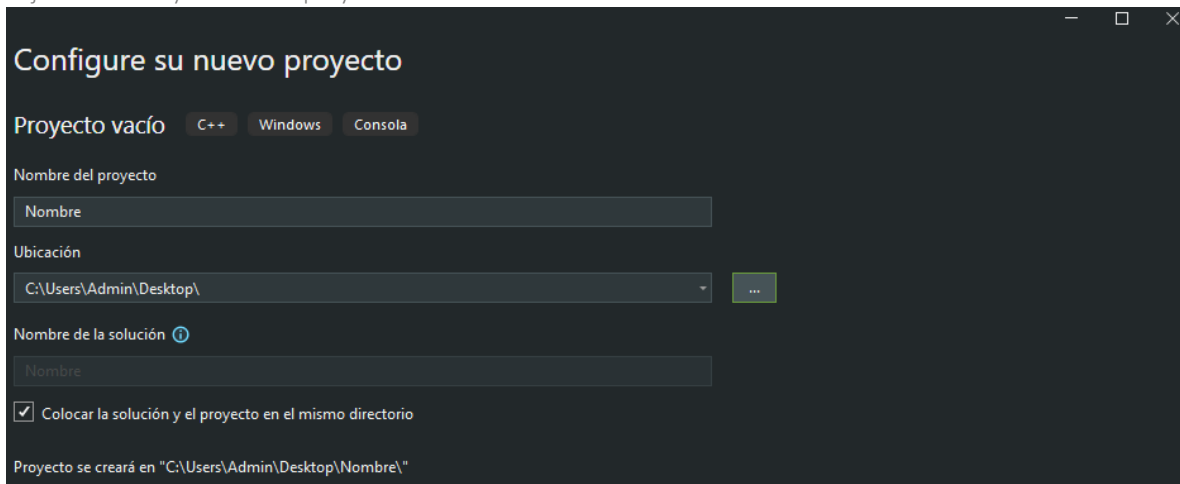
# Inicio básico

Esta API de encabezado no compilado requiere ser usada en un entorno Visual Studio debido al uso de caracteres Unicode en identificadores de variables.

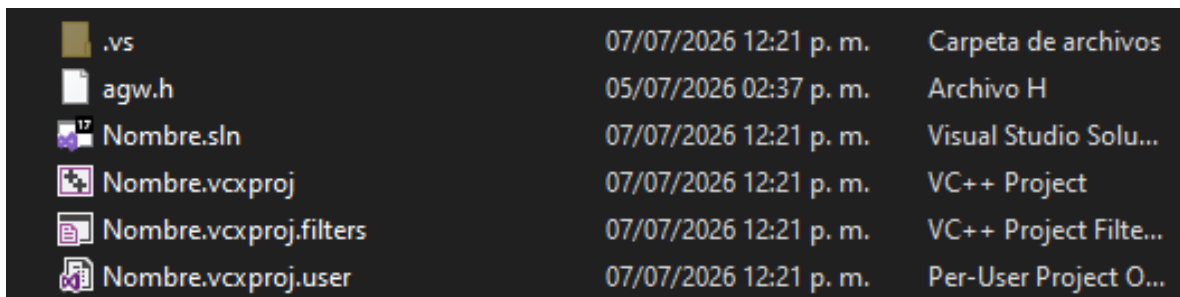
Para comenzar se requiere crear una nueva solución vacía según la siguiente guía de imágenes:



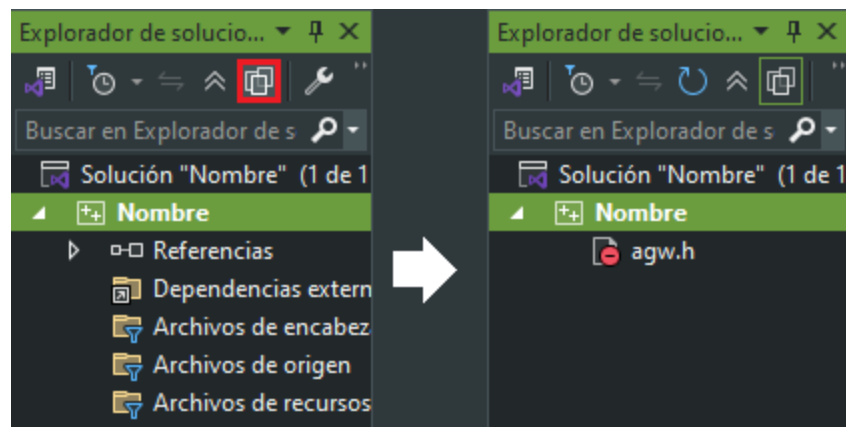
Elija el nombre y ruta de su proyecto:



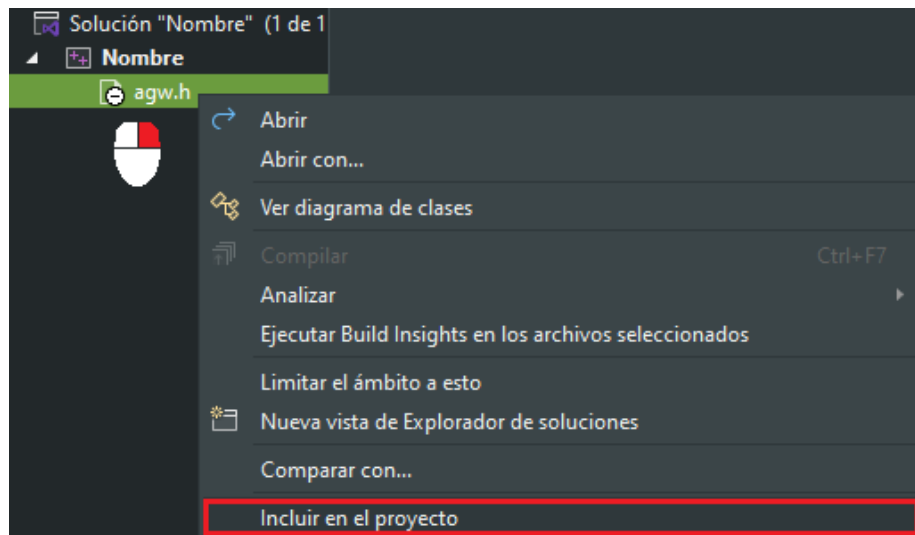
Copie AGW a la ruta de su Solución:



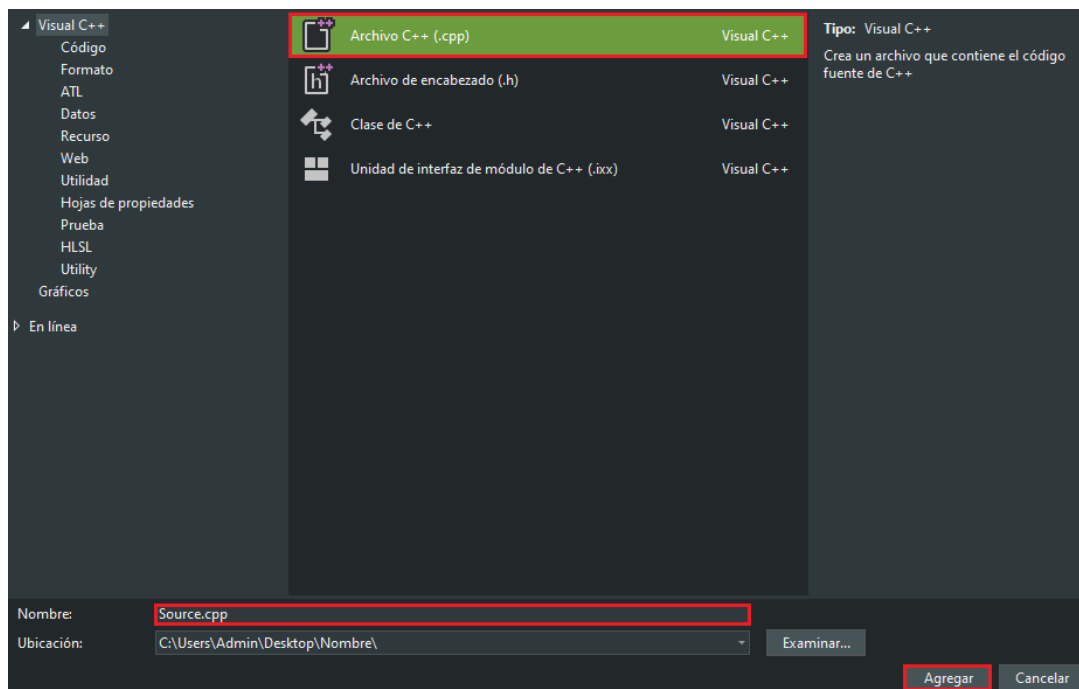
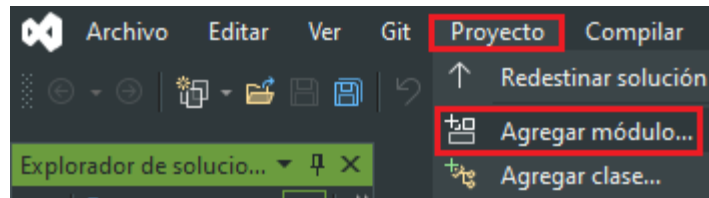
Configure su Solución para mostrar los archivos de su directorio:



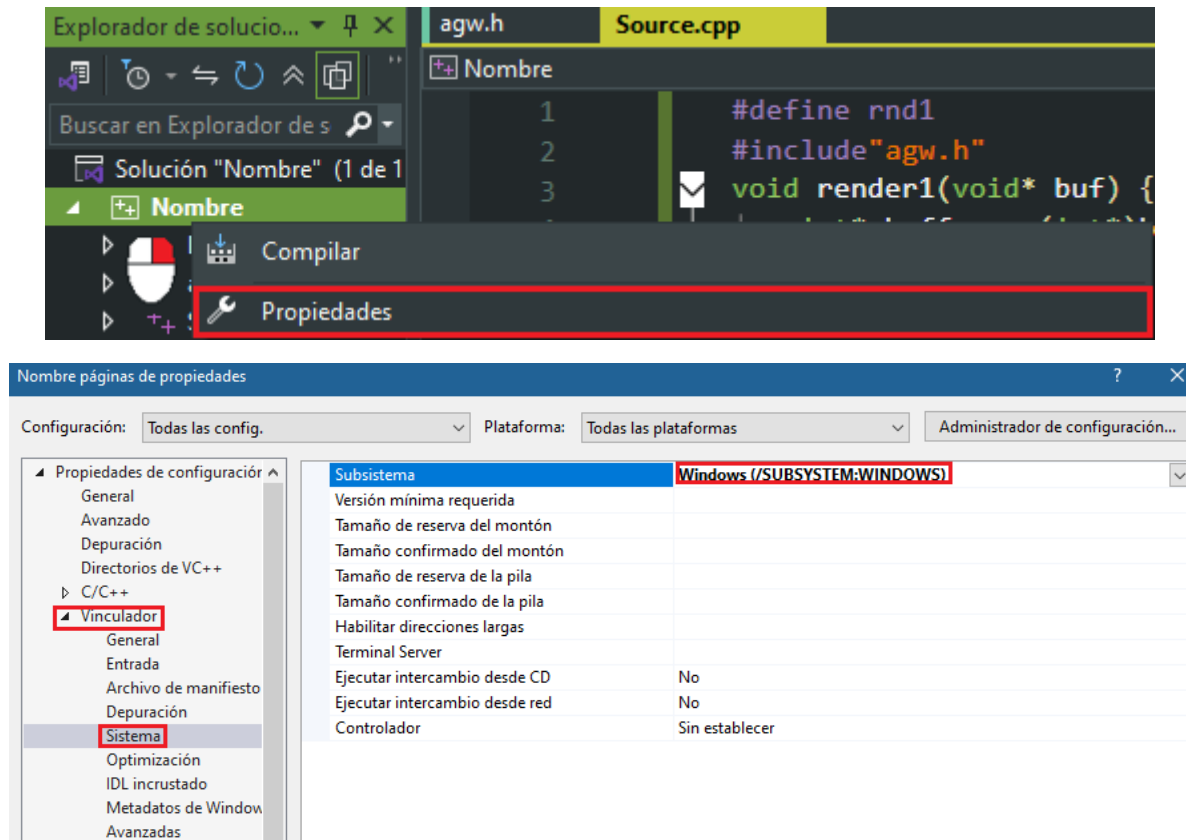
Después, incluya AGW en el proyecto:



Y agregue un archivo de código fuente:



Por último, necesita configurar su solución para aplicaciones de ventana gráfica:



# Rutina de inicio

La rutina de inicio es el conjunto de funciones estándar que se definen para que AGW pueda compilar y funcionar correctamente, la primera función es:

```
vent* setup(vent* v) {  
  
    v->color_borrado = 0x00000000; //(opcional)  
  
    v->tamaño_x = 640;  
  
    v->tamaño_y = 480;  
  
    v->pos_x = 0;  
  
    v->pos_y = 0;  
  
    v->titulo = L"prueba";  
  
    return v;  
  
}
```

Que está definida en el núcleo de AGW, toma y devuelve un puntero a un struct vent, tipo propietario de la API, el cual contiene propiedades de la ventana:

```
struct vent {  
  
    const wchar_t* titulo;  
  
    int tamaño_x;  
  
    int tamaño_y;  
  
    int pos_x;  
  
    int pos_y;  
  
    int color_borrado;  
  
    HFONT fuente = NULL;  
  
};
```

Pero AGW solo requiere tener una ventana con un ancho mayor a 0 y con un titulo para funcionar, de lo contrario, al intentar abrirlo solo se cerrará inmediatamente como mecanismo de seguridad.

Opcionalmente, se puede almacenar el puntero obtenido a vent para obtener las propiedades de la ventana en tiempo real, como su posición, y si se quisiera hacer un cambio de tamaño a la ventana, se tendría que actualizar el campo correspondiente antes de hacerlo.

# Macros de inclusión de contenido

Las macros de inclusión de contenido permiten utilizar y excluir contenido en tiempo de compilación para hacer mas ligero el ejecutable de salida, estas se colocan antes de incluir el encabezado de AGW, a continuación, una lista con todas las macros implementadas y su función:

`#define SDL_2` es una macro en versión de prueba que posteriormente se usara para cambiar la API base, de GDI a SDL2.

`#define BOTONES` es una macro completamente funcional que implementa un servicio de interfaz con botones completamente funcional, el cual permite los modos de operación descritos por las macros que inician con `BT_...`

`#define SLDR` es una macro funcional que implementa la gestión, manejo y control de deslizadores, el cual tiene modos de operación descritos por las macros que inician por `SLID_...`

`#define MAX` es una macro del núcleo que esta incompleta, es insegura y no está en remodelación activa, debido a que sus usos son limitados y fácilmente logrados con otras funciones implementadas de forma externa, esta macro implementa el botón de maximizar de la barra dinámica superior.

`#define msg_` es una macro del núcleo que permite llamar a un hook/callback bajo el formato de Windows para interceptar o atender mensajes que la API, como estándar ligero y mínimo que es, no implementa ni atiende.

`#define rnd_set1` es una macro base que permite implementar la función de callback de inicialización de hilo numero 1, lo cual es útil para inicializar objetos o funciones que no son thread-safe, o para realizar acciones en el hilo correspondiente sin hacerlo en el bucle temporizado.

`#define rnd_set2` es el equivalente de `rnd_set1` que se aplica al hilo de render número 2.

`#define borrador` es una macro base que borra el framebuffer con ayuda de ambos hilos preestablecidos, cada 16ms, pero se desaconseja su uso en aplicaciones serias, porque los hilos se inician naturalmente descoordinados, para evitar perder ciclos solo por sincronizarlos.

`#define rnd1` permite llamar a la función callback estándar del hilo de renderizado número 1, cada 16ms.

`#define rnd2` es el equivalente funcional a `rnd1`, con la ligera distinción de que usualmente el hilo 2 ejecuta menos instrucciones del núcleo por ciclo funcional, porque no atiende a las sub-APIs de botones ni de deslizadores.

`#define proces` es una macro que permite utilizar al hilo de arranque como hilo de procesamiento general, sin salida a framebuffer, que también esta ajustado para hacer un ciclo cada 8ms.

`#define precis` es una macro del núcleo que permite forzar los ticks del sistema a 1ms para mejorar la temporización interna

# Manejo de subrutinas de renderizado

La principal característica de AGW es que su salida de video es en formato de framebuffer plano, lo que significa que solo el programador sabe cual es el formato correcto, lo que hace posible hacer optimizaciones tales como llenar la pantalla de color solido (o con patrones) con un solo bucle, en vez de usar bucles anidados, que son en la mayoría de casos, más ineficientes.

La forma estándar de dibujar en el buffer de video en AGW es mediante los punteros enviados a las funciones de render, que son de tipo void, las 2 funciones de render base son:

```
void render1(void* buffer);
```

```
void render2(void* buffer);
```

y solo se habilitan al momento de definir la macro correspondiente:

```
#define rnd1
```

```
#define rnd2
```

Dentro de cualquiera de ambos callbacks la rutina estándar para obtener los buffers de dibujo útiles es:

```
void render1(void* buf) {  
    int* buffer = (int*) buf;  
}
```

# Registro y modificación de deslizadores

La sub API de deslizadores, descrita por #define SLDR permite registrar controles deslizantes de interfaz de usuario mediante la función propietaria:

```
(int) regis_slider (SLID_FLAG, pos_x, pos_y, ancho, alto, num_min, num_max, *float val);
```

La cual retorna un valor entero de 32 bits con el identificador numero del elemento, necesario para su modificación, la cual, se realiza con la siguiente función propietaria:

```
(void) mod_slider (ID, SLID_FLAG, pos_x, pos_y, ancho, alto, num_min, num_max, *float val);
```

Los posibles flags de comportamiento son los siguientes:

SLID\_VISIBLE el cual permite hacer visible el control deslizante, lo que permite añadir, o encimar un estilo propio sin gastar potencia

SLID\_INFO muestra un cuadro de 30\*18 pixeles con el valor actual

SLID\_HABILIT habilita el slider, de lo contrario, no se dibuja la línea de color, ni se permite la interacción

SLID\_ROJO provoca que la línea de color sea roja

SLID\_AZUL provoca que la línea de color sea azul

SLID\_VERDE provoca que la línea de color sea verde

SLID\_HORIZONTAL permite que el control deslizante este orientado en posición horizontal en vez de vertical

Peculiaridades:

Para la correcta apariencia y la eliminación de errores de segmentación, el control deslizante, tiene una diferencia de 16 pixeles en tamaño, 8 arriba y 8 debajo, si bien esto no modifica el rango, es algo a tomar en cuenta si se planea agregar un estilo sobre el deslizador reciclando variables internas de la API

El cuadro de información de SLID\_INFO es de tamaño fijo, y esta anclado a la izquierda del control deslizante, pero sobresale 22 pixeles hacia abajo tanto horizontal como verticalmente (SLID\_HORIZONTAL)

# Registro y modificación de botones

La sub API de botones, descrita por `#include BOTONES` permite registrar botones de interfaz de usuario mediante la función propietaria:

```
(Int) regis_boton (char atajo, char tipo, unsigned int x, unsigned int y, unsigned int ancho, unsigned int alto, void activo(void*), void desactivo(void*), void* arg_act, void* arg_suelt);
```

La cual tiene, en orden de aparición, los siguientes argumentos:

`char atajo`: permite dar un carácter (") o un VK de Windows para asignar una tecla atajo

`char tipo`: permite especificar que flags de operación se desean, de entre 8 diferentes

`unsigned int x`: es la posición horizontal física del botón en pantalla

`unsigned int y`: es la posición vertical física del botón en pantalla

`unsigned int ancho`: es el ancho del botón

`unsigned int alto`: es el alto del botón

`void activo(void*)`: es un callback que se llama al activar el botón que debe tener la firma `void <nombre> (void* <arg>)`

`void desactivo(void*)`: es un callback que se llama al desactivar el botón, y requiere la misma firma que el callback de activación

`void* arg_act`: es el argumento que se pasa al callback de activación, este puede ser un puntero a cualquier estructura, objeto o variable

`void* arg_suelt`: es el argumento que se pasa al callback de desactivación, este tiene el mismo formato que el de `arg_act`

Y la función retorna un identificador que se pasa al modificar el botón:

```
void mod_boton (int button_handler, char atajo, char tipo, unsigned int x, unsigned int y, unsigned int ancho, unsigned int alto, void activo(void*), void desactivo(void*), void* arg_act, void* arg_suelt);
```

Los flags que permite la sub API de botones son los siguientes:

BT\_ACTIVE Si se pasa al registrar un botón, este tendrá aspecto de presionado al estar en reposo, y aspecto de reposo al estar presionado

BT\_SHORTCUT utiliza el argumento de carácter de regis\_boton\_ex para accionar el botón, sin animarlo

BT\_DRAW dibuja el botón independientemente de si está activo, inactivo, habilitado o inhabilitado

BT\_DELAY permite el tiempo de espera bloqueante al accionar con teclas, y con mouse usando el flag

BT\_REPEAT\_ON\_CLICK

BT\_REPEAT\_ON\_CLICK repite la acción de pulsar y despulsar al hacer click con el mouse, sin animar el botón, NO APLICA A MODO SWITCH

BT\_ANIM habilita la animación del botón ante pulsaciones físicas del botón con el mouse, NO APLICA A MODO SWITCH, NO APLICA CON BT\_REPEAT\_ON\_CLICK

BT\_PUSH si se utiliza, el botón se apagará al dejar de hacer click, o al abandonar la casilla del botón, si no, se comportara como un botón biestable

BT\_HABILIT habilita el botón para que sea pulsable, animable y utilizable, si no se usa, el botón será gris, sin animación, y de aspecto fijo

# Este manual solo aplica para la versión

1.3.0 de AGW según el sistema de versionado interno

©2026 Netomaster8900

Este manual, junto con el código fuente se distribuyen  
bajo la licencia MIT